MACQUARIE University

COMP8221 Advanced Machine Learning

GNN framework, augmentation and training

Zitai Qiu, School of Computing, zitai.qiu@mq.edu.au



Outline

- Modern deep learning tricks
- Layer connectivity:
  - Number of layers
  - Skip connections
- Graph Manipulation
- Different task level
- Loss Function & Evaluation Metrics
- Data Split

Recap: Graph Setup

MACQUARIE University

Assume we have a graph:

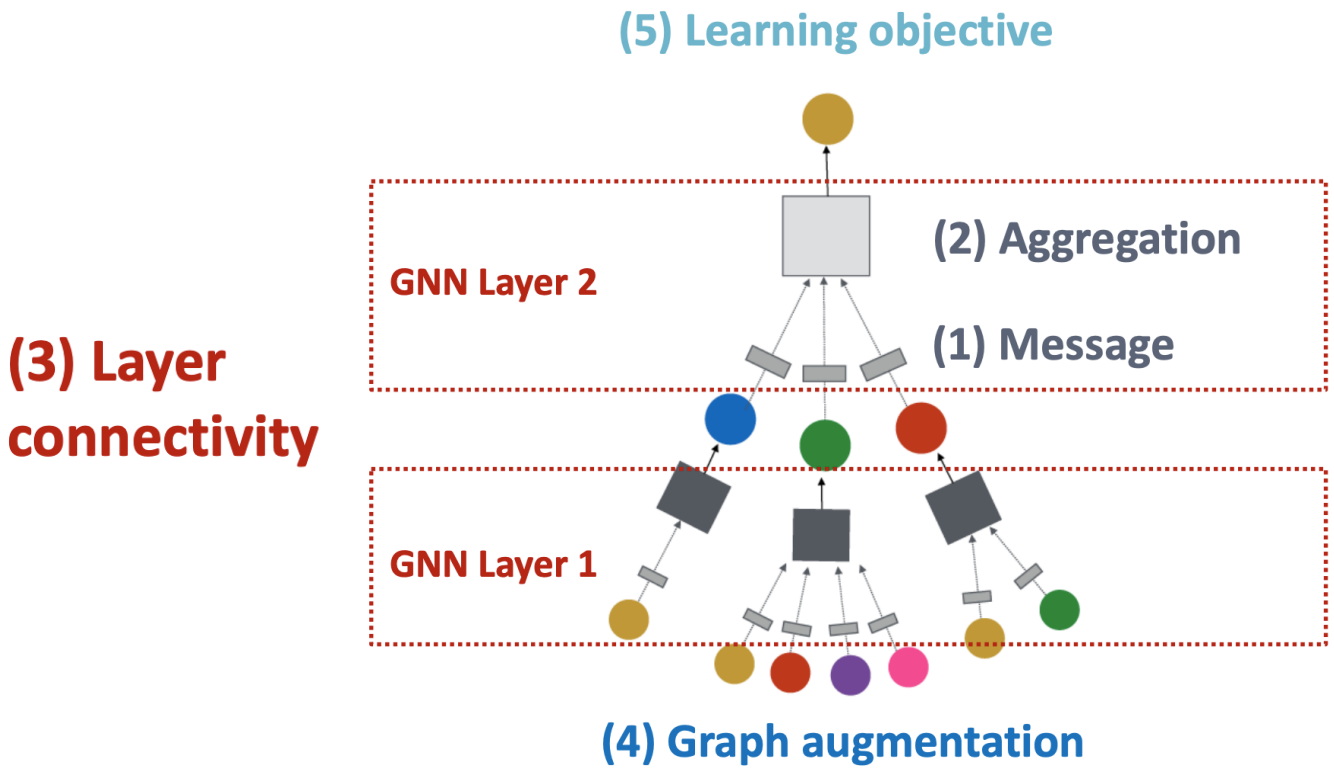
- $G = (V, E)$
- V is the vertex set
- E is the edge set (undirected edge & directed edge)
- $\boldsymbol{A}$ is the Adjacency matrix
- $\boldsymbol{X} \in R^{|V| \times m}$ is a matrix of node features
- v represents a node in V
- $N(v)$ is the set of neighbors of v

- Examples of node features:

Social networks: User profile, User image, …

Biological networks: Gene expression profile, Gene functional information, …

Recap: General GNN Framework



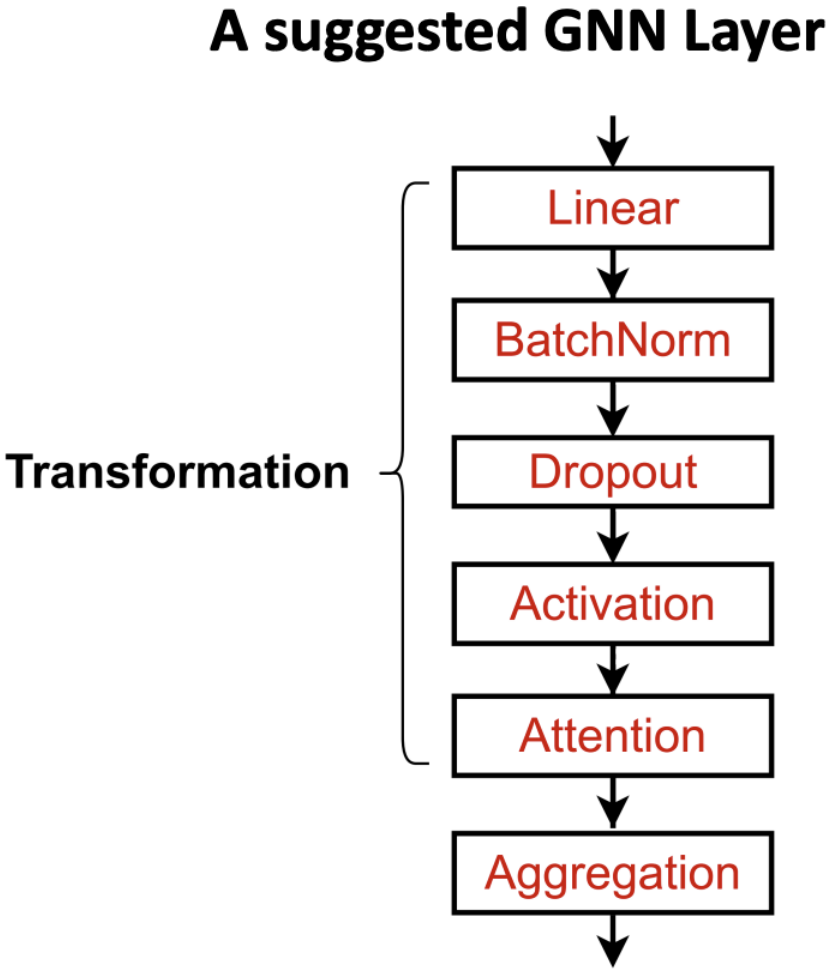
GNN layer = Message + Aggregation

Different instantiations under this perspective, such as GCN, GraphSAGE, GAT, …

GNN layer in Practice

MACQUARIE University

- In practice, these classic GNN layers are a great starting point
- We can often get better performance by considering a general GNN layer design
- Concretely, we can include modern deep learning modules that proved to be useful in many domains



GNN layer in Practice

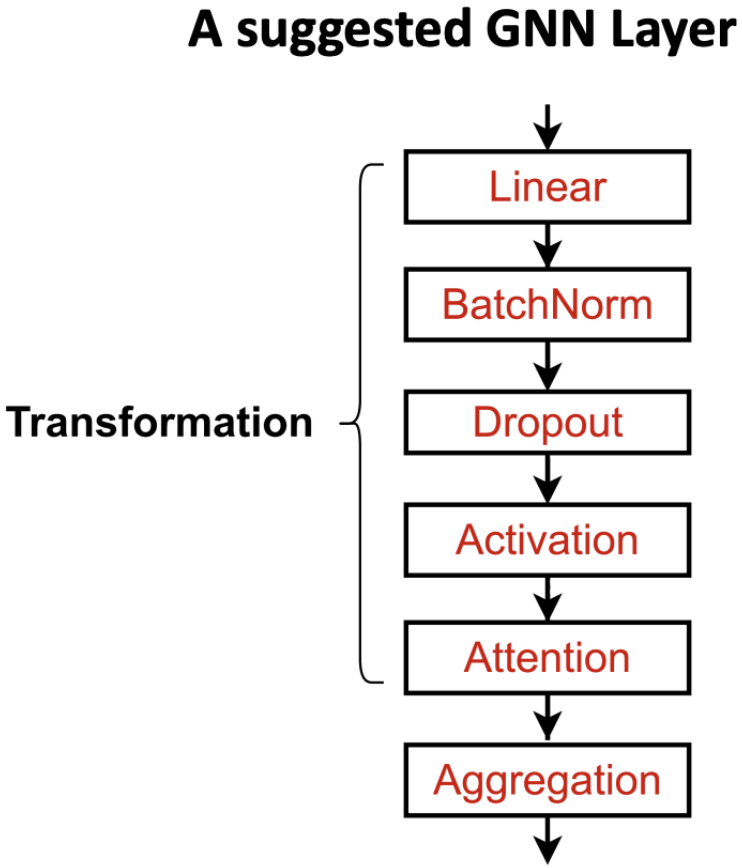
MACQUARIE University

- Many modern deep learning modules can be incorporated into a GNN layer

- **Batch Normalization**:
  Stabilize neural network training

- **Dropout**:
  Prevent overfitting

- **Attention/Gating**:
  Control the importance of a message



Batch Normalization

- Goal: Stabilize neural networks training
- Idea: Given a batch of inputs (node embeddings)

  Re-center the node embeddings into zero mean

  Re-scale the variance into unit variance

Input: $\mathbf{X} \in \mathbb{R}^{N \times D}$

N node embeddings

Trainable Parameters:

$\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{D}$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times D}$

Normalized node embeddings

Step 1:

Compute the mean and variance over N embeddings

$$\boldsymbol{\mu}_j = \frac{1}{N}\sum_{i=1}^{N} \mathbf{X}_{i,j}$$

$$\boldsymbol{\sigma}_j^2 = \frac{1}{N}\sum_{i=1}^{N} \left(\mathbf{X}_{i,j} - \boldsymbol{\mu}_j\right)^2$$

Step 2:

Normalize the feature using computed mean and variance

$$\widehat{\mathbf{X}}_{i,j} = \frac{\mathbf{X}_{i,j} - \boldsymbol{\mu}_j}{\sqrt{\boldsymbol{\sigma}_j^2 + \epsilon}}$$

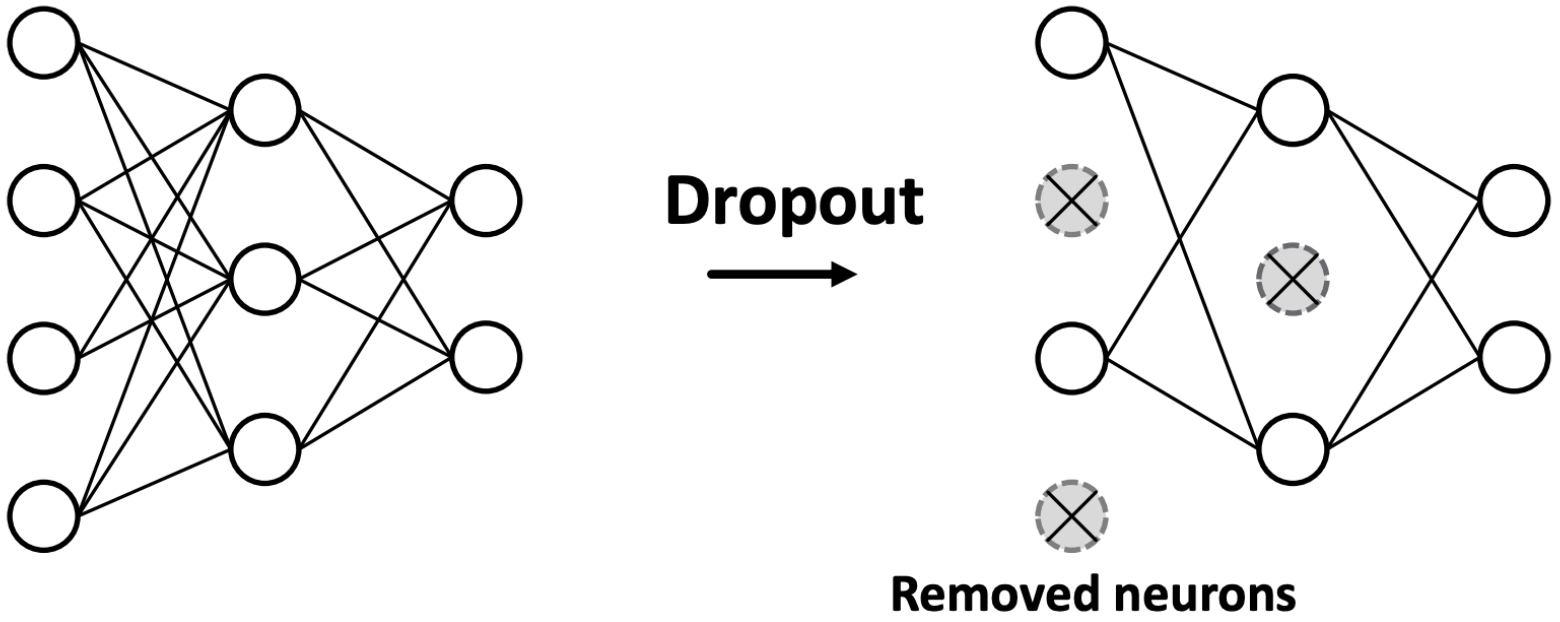
$$\mathbf{Y}_{i,j} = \boldsymbol{\gamma}_j \widehat{\mathbf{X}}_{i,j} + \boldsymbol{\beta}_j$$

Dropout

- Goal: Regularize a neural net to prevent overfitting.
- Idea:

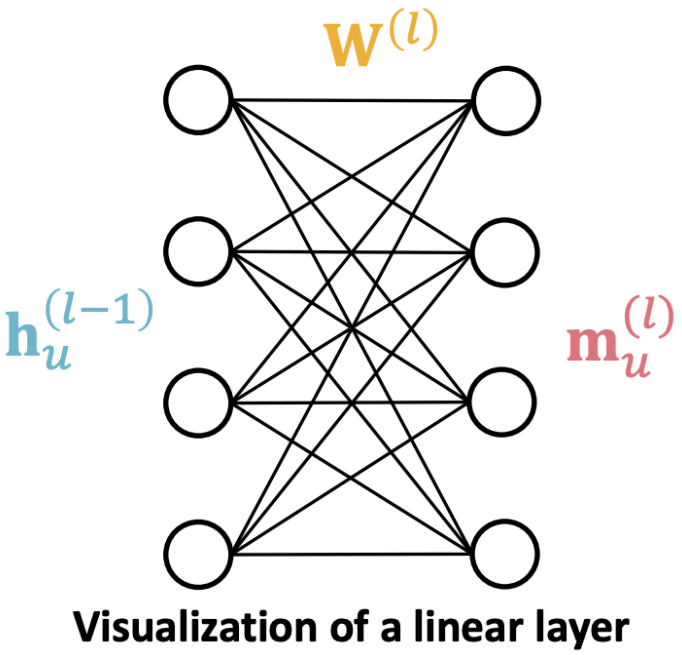
  During training: with some probability p, randomly set neurons to zero (turn off)

  During testing: Use all the neurons for computation



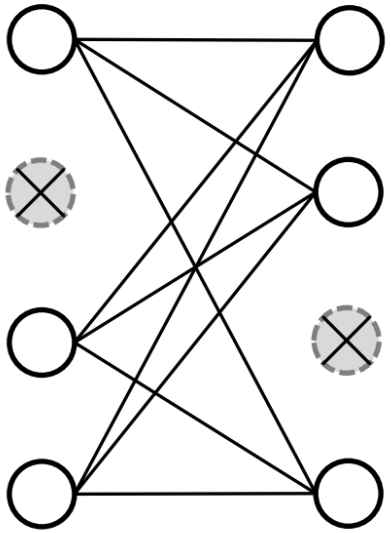
Dropout for GNNs

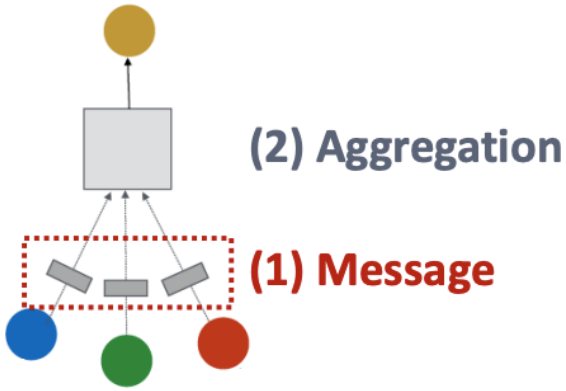
- In GNN, Dropout is applied to the linear layer in the message function
- A simple message function with linear layer: $m_u^{(l)} = W^{(l)} h_u^{(l-1)}$



Visualization of a linear layer

Dropout

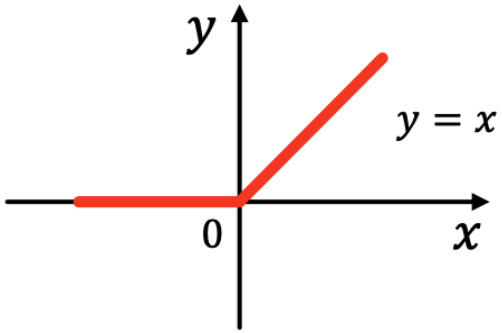




Activation: Non-linearity

- **ReLU**:

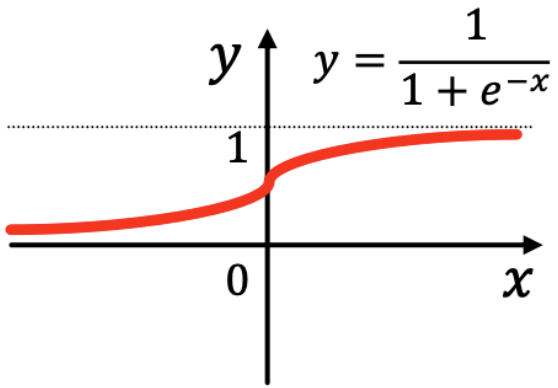
  Most commonly used



- **Sigmoid**:

  $\sigma(x_i) = \frac{1}{1+e^{-x_i}}$

  Used only when you want to restrict the range of your embeddings



- tanh, Parametric ReLU, …,

GNN Layer in Practice

MACQUARIE University

- Summary: Modern deep learning modules can be included into a GNN layer for better performance

Designing novel GNN layers is still an active research frontier!

You can explore diverse GNN designs or try out your own ideas in practical workshops using python and very helpful packages

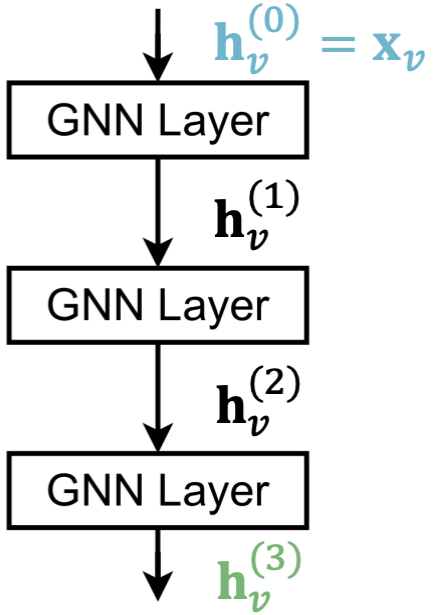
Stacking GNN Layers

- How to construct a Graph Neural Network?

  The standard way: Stack GNN layers sequentially

  Input: Initial raw node feature x_v

  Output: Node embeddings $h_v^{(L)}$ after L GNN layers



The Over-smoothing Problem

MACQUARIE University

- The issue of stacking many GNN layers

  GNN suffers from the over-smoothing problem

- The over-smoothing problem:

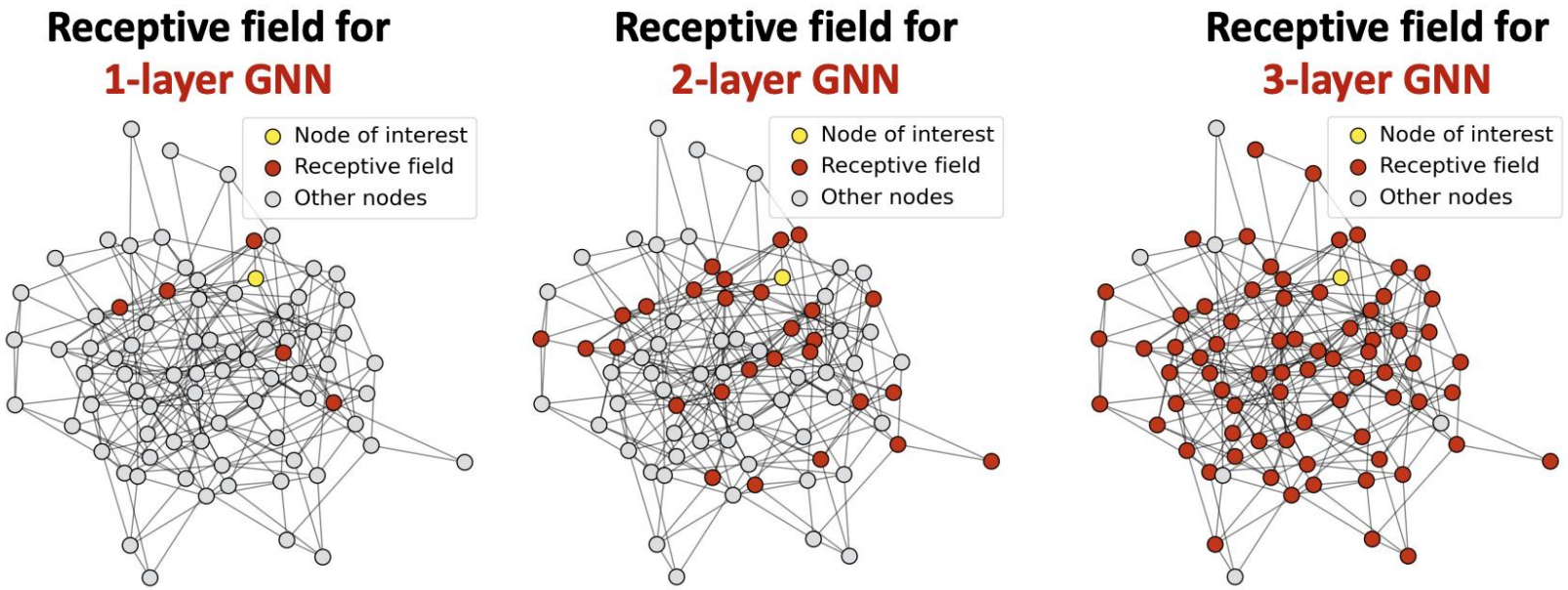
  All the node embeddings converge to the same value

  This is bad because we want to use node embeddings to differentiate nodes

- Why does the over-smoothing problem happen?

Receptive Field of a GNN

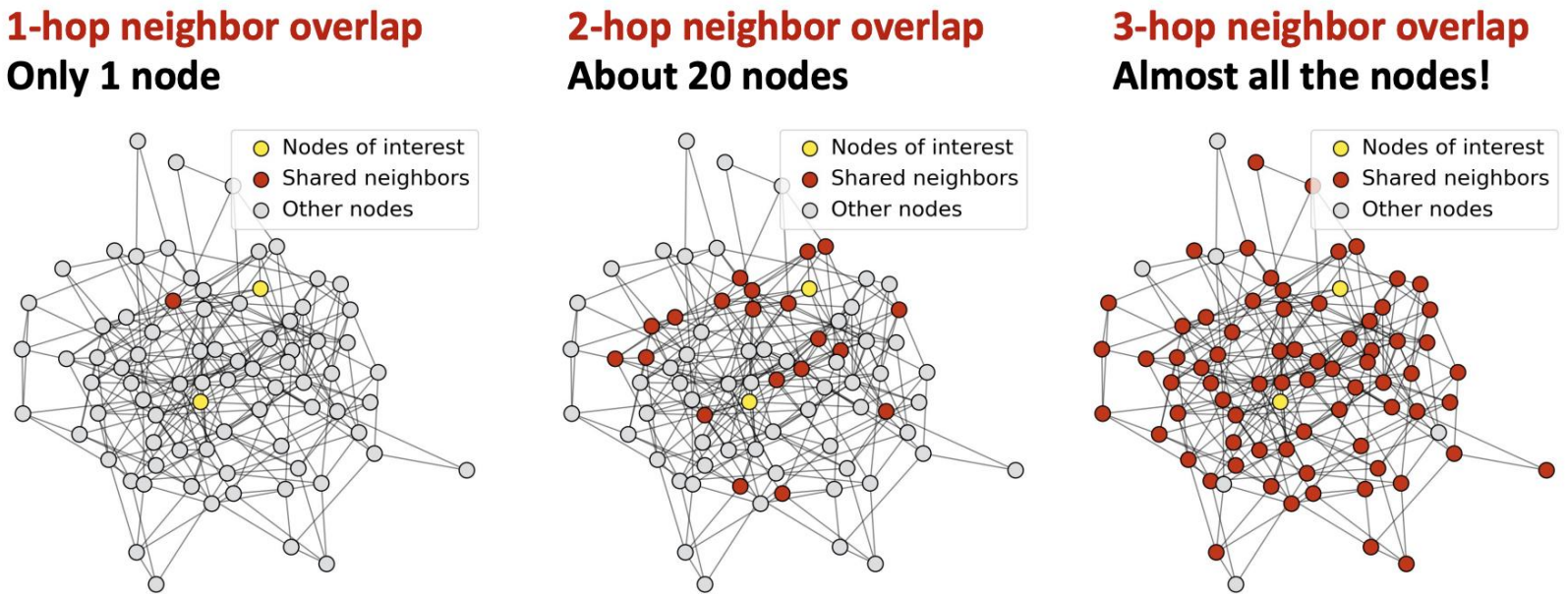
- **Receptive field**:
  The set of nodes that determine the embedding of a node of interest
- In a K-layer GNN, each node has a receptive field of K-hop neighborhood.



Receptive Field of a GNN

- Receptive field overlap for two nodes

  The shared neighbours quickly grows when we increase the number of GNN layers



Receptive Field & Over-smoothing

MACQUARIE University

- We can explain over-smoothing via the notion of the receptive field

  We know the embedding of a node is determined by its receptive field.

  If two nodes have highly-overlapped receptive fields, then their embeddings are highly similar

Logic:

- Stack many GNN layers → nodes will have highly overlapped receptive fields → node embeddings will be highly similar → suffer from the over-smoothing problem

Next: how do we overcome over-smoothing problem?

Layer Connectivity

MACQUARIE University

- Be cautious when adding GNN layers

  Unlike neural networks in other domains (CNN for image classification), **adding more GNN layers do not always help.**

- Do not set GNN layer L to be unnecessarily large!

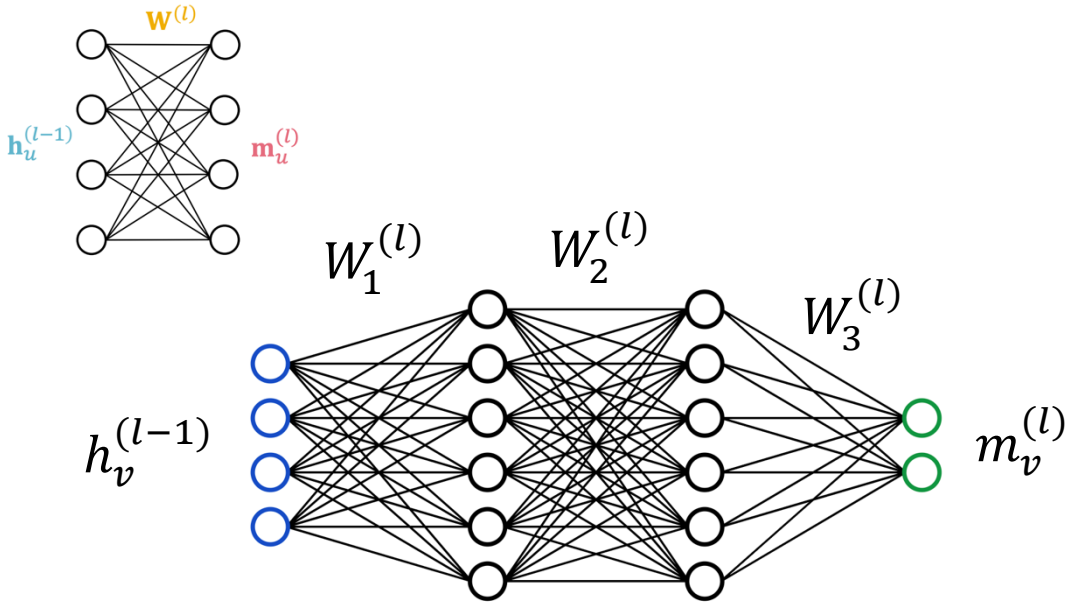
- Question: How to **enhance the expressive power** of a GNN, if the number of GNN layers is small (Shallow GNNs)?

Expressive power for Shallow GNNs

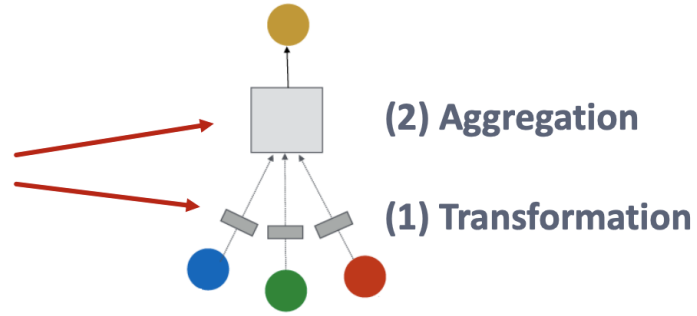
- **How to make a shallow GNN more expressive?**
- **Solution 1**: Increase the expressive power within each GNN layer

In our previous examples, each transformation or aggregation function only include one linear layer

We can make aggregation / transformation become a deep neural network



If needed, each box could include a 3-layer MLP



Expressive power for Shallow GNNs

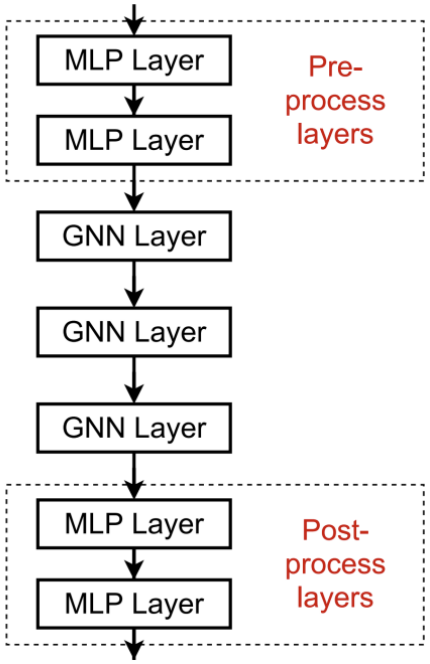


MACQUARIE University

- **How to make a shallow GNN more expressive?**
- **Solution 2:** Add layers that do not pass messages

  A GNN does not necessarily only contain GNN layers

  E.g., we can add MLP layers (applied to each node) before and after GNN layers, as pre-process layers and post-process layers



Pre-processing layers: Important when encoding node features is necessary.
E.g., when nodes represent images/text

Post-processing layers: Important when reasoning / transformation over node embeddings are needed
E.g., graph classification, knowledge graphs

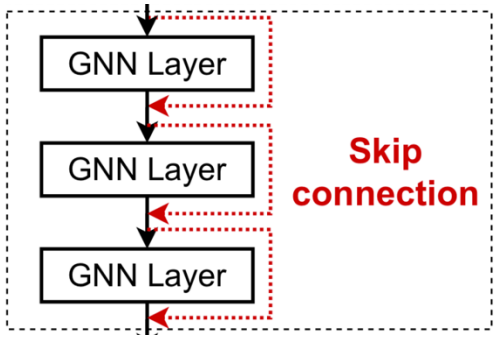
In practice, adding these layers works great!

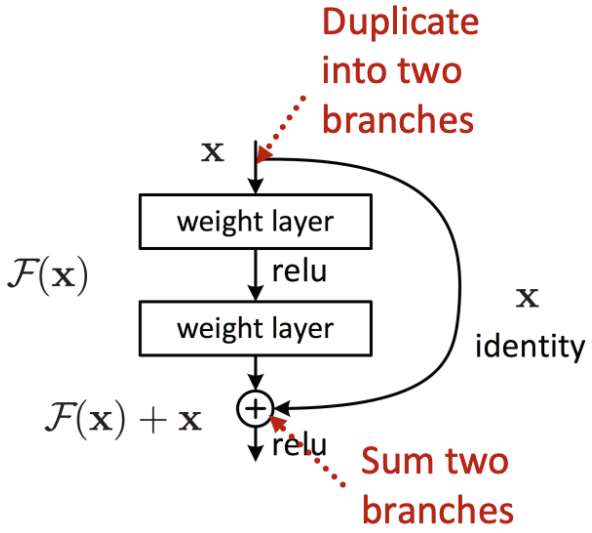
Layer Connectivity

- **What if my problem still requires many GNN layers?**
- **Solution3**: Add skip connections in GNNs

  Node embeddings in earlier layers can sometimes better differentiate nodes

  We can increase the impact of earlier layers on the final node embeddings, by adding shortcuts in GNN





Idea of skip connections:

Before adding shortcuts:

$$\boldsymbol{F}(\mathbf{x})$$

After adding shortcuts:

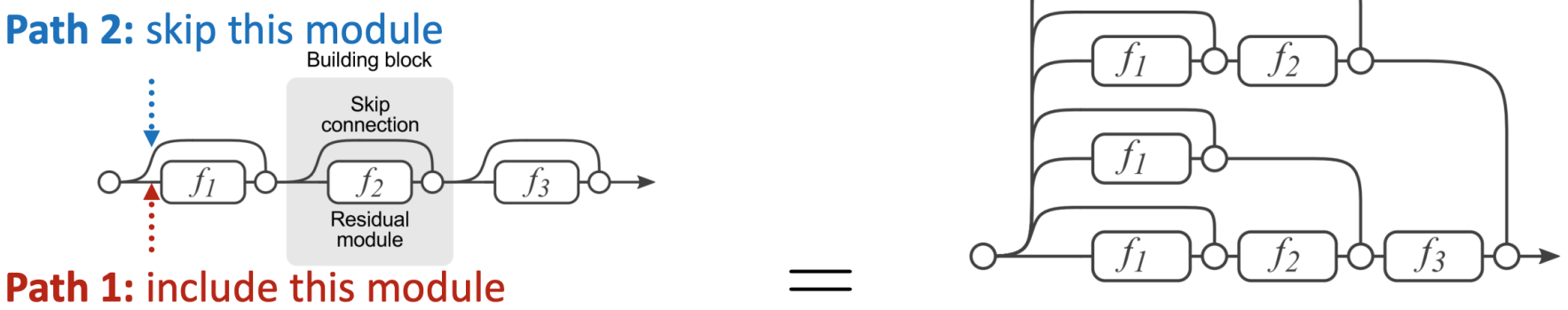
$$\boldsymbol{F}(\mathbf{x}) + \mathbf{x}$$

Skip Connections

- Intuition: Skip connections create a mixture of models

  N skip connections → 2^N possible paths

  We automatically get **a mixture of shallow GNNs and deep GNNs**



Example: GCN with Skip Connection

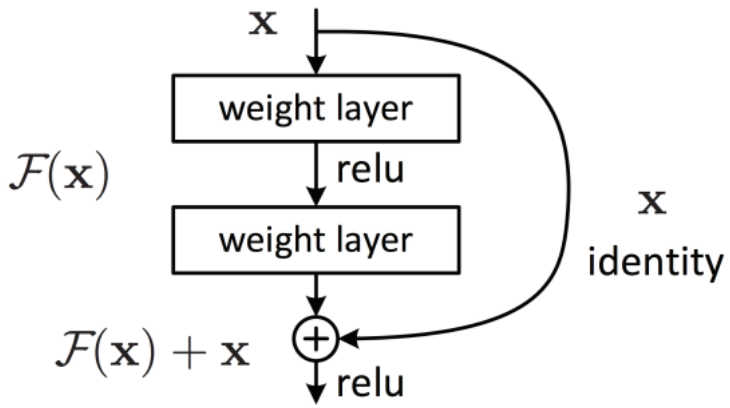
MACQUARIE University

- A standard GCN layer:
- $h_v^{(l)} = \sigma(\sum_{u \in N(v)} W^{(l)} \frac{h_u^{(l-1)}}{|N(v)|})$

This is our $F(x)$

- One GCN layer with skip connection:
- $h_v^{(l)} = \sigma(\sum_{u \in N(v)} W^{(l)} \frac{h_u^{(l-1)}}{|N(v)|} + h_v^{(l-1)})$

$F(x)$ + x



- If we skip two layers, what is $h_v^{(l)}$?

Graph Manipulation

MACQUARIE University

- **So far, we assume:**

  Raw input graph = computational graph

- In practice, it's unlikely that the input graph happens to be the optimal computation graph for embeddings:

  The input graph lacks features → ?
  The graph is too sparse → ?
  The graph is too dense → ?
  The graph is too large → ?

Graph Manipulation

MACQUARIE University

- **So far, we assume:**

  Raw input graph = computational graph

- In practice, it's unlikely that the input graph happens to be the optimal computation graph for embeddings:

  The input graph lacks features → need augmented features
  The graph is too sparse → inefficient message passing
  The graph is too dense → message passing is too costly
  The graph is too large → cannot fit the computational graph into a GPU

Graph Manipulation

MACQUARIE University

- **How can we resolve the problems?**

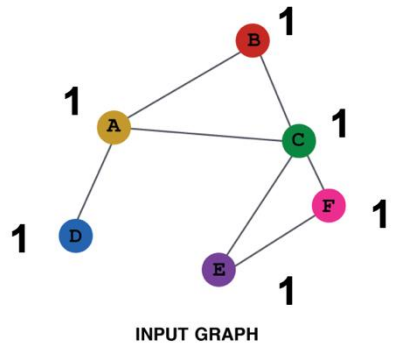
  The input graph lacks features → Feature augmentation
  The graph is too sparse → Add virtual nodes / edges
  The graph is too dense → Sample neighbors when doing message passing
  The graph is too large → Sample subgraphs to compute embeddings

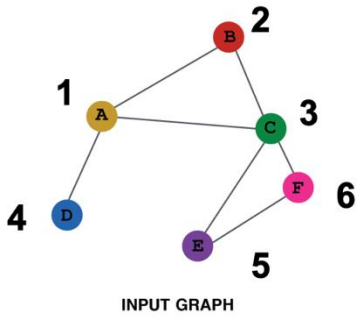
Feature Augmentation

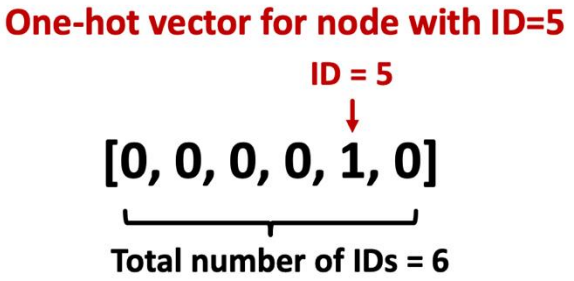
- Input graph does not have node features.
- This is common when we only have the adj. matrix.

- **Standard approach**:
  1) Assign constant values to nodes
  2) Assign unique IDs to nodes

     These IDs are converted into **one-hot vectors**

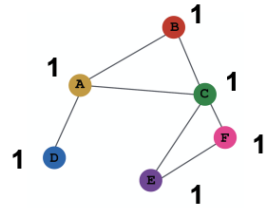
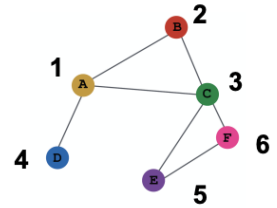






Feature Augmentation

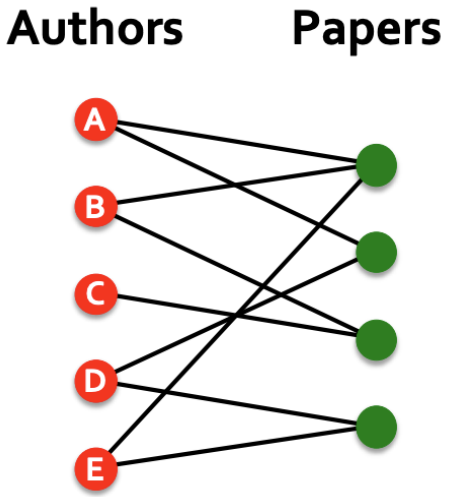
MACQUARIE University

	Constant node feature	One-hot node feature
	 INPUT GRAPH	 INPUT GRAPH
Expressive power	**Medium**. All the nodes are identical, but GNN can still learn from the graph structure	**High**. Each node has a unique ID, so node-specific information can be stored
Inductive learning (Generalize to unseen nodes)	**High**. Simple to generalize to new nodes: we assign constant feature to them, then apply our GNN	**Low**. Cannot generalize to new nodes: new nodes introduce new IDs, GNN doesn't know how to embed unseen IDs
Computational cost	**Low**. Only 1 dimensional feature	**High**. High dimensional feature, cannot apply to large graphs
Use cases	Any graph, inductive settings (generalize to new nodes)	Small graph, transductive settings (no new nodes)

Add Virtual Nodes / Edges

- **Motivation:** The graph is too sparse.
- **Common approach:**
  1) Connect 2-hop neighbors via virtual edges

- **Example**:
  Author-to-papers (they authored)
  2-hop virtual edges make an **author-author collaboration graph**



Add Virtual Nodes / Edges

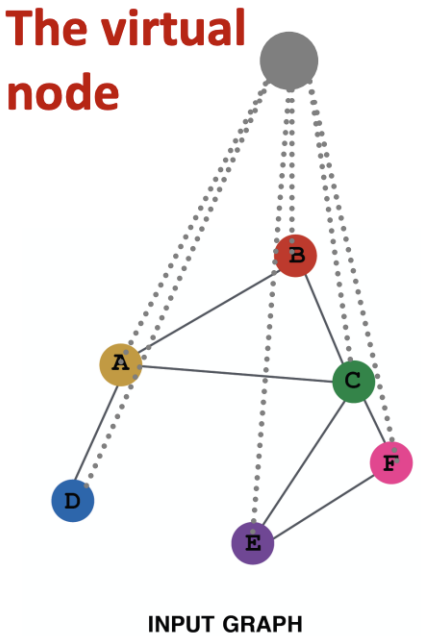
- **Motivation:** The graph is too sparse.
- **Common approach:**

  2) Add virtual nodes

  The virtual node will connect to all the nodes in the graph

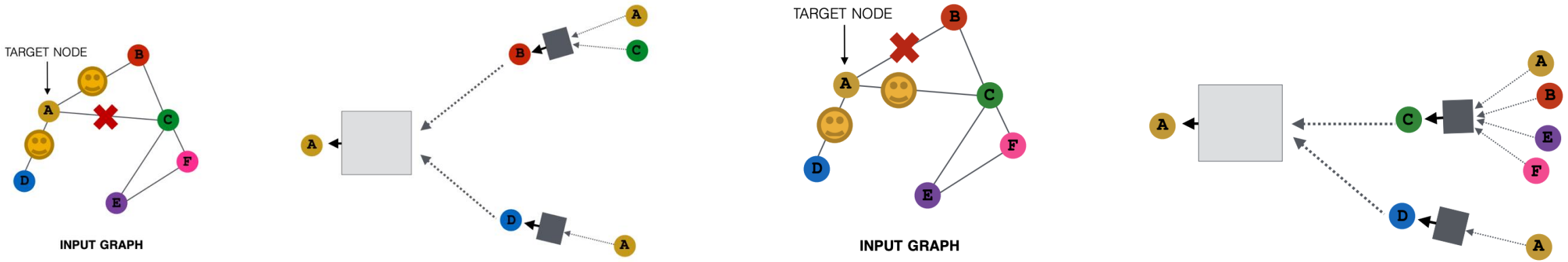
Suppose in a sparse graph, two nodes have shortest path distance of 10. After adding the virtual node, all the nodes will have a distance of 2.

Benefits: Greatly improves message passing in sparse graphs



Node neighbourhood sampling

- For **dense/large graphs, high-degree nodes**
- Idea: **Randomly** determine a node's neighborhood for message passing
- For example, we can randomly choose 2 neighbors to pass messages
- Each time, different nodes may be sampled for training

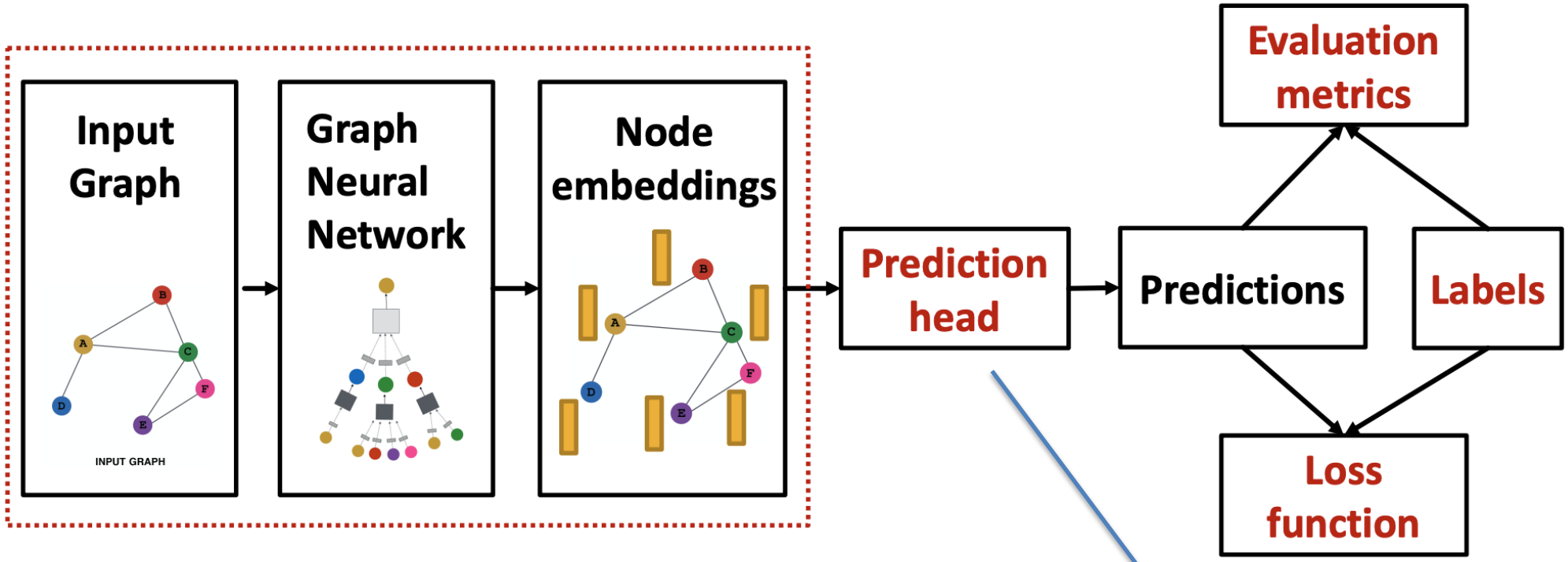


- Get embeddings **similar to** the case where all the neighbors are used.
- **Benefits: Greatly reduce computational cost**

GNN Training Pipeline

MACQUARIE University

So far what we have covered

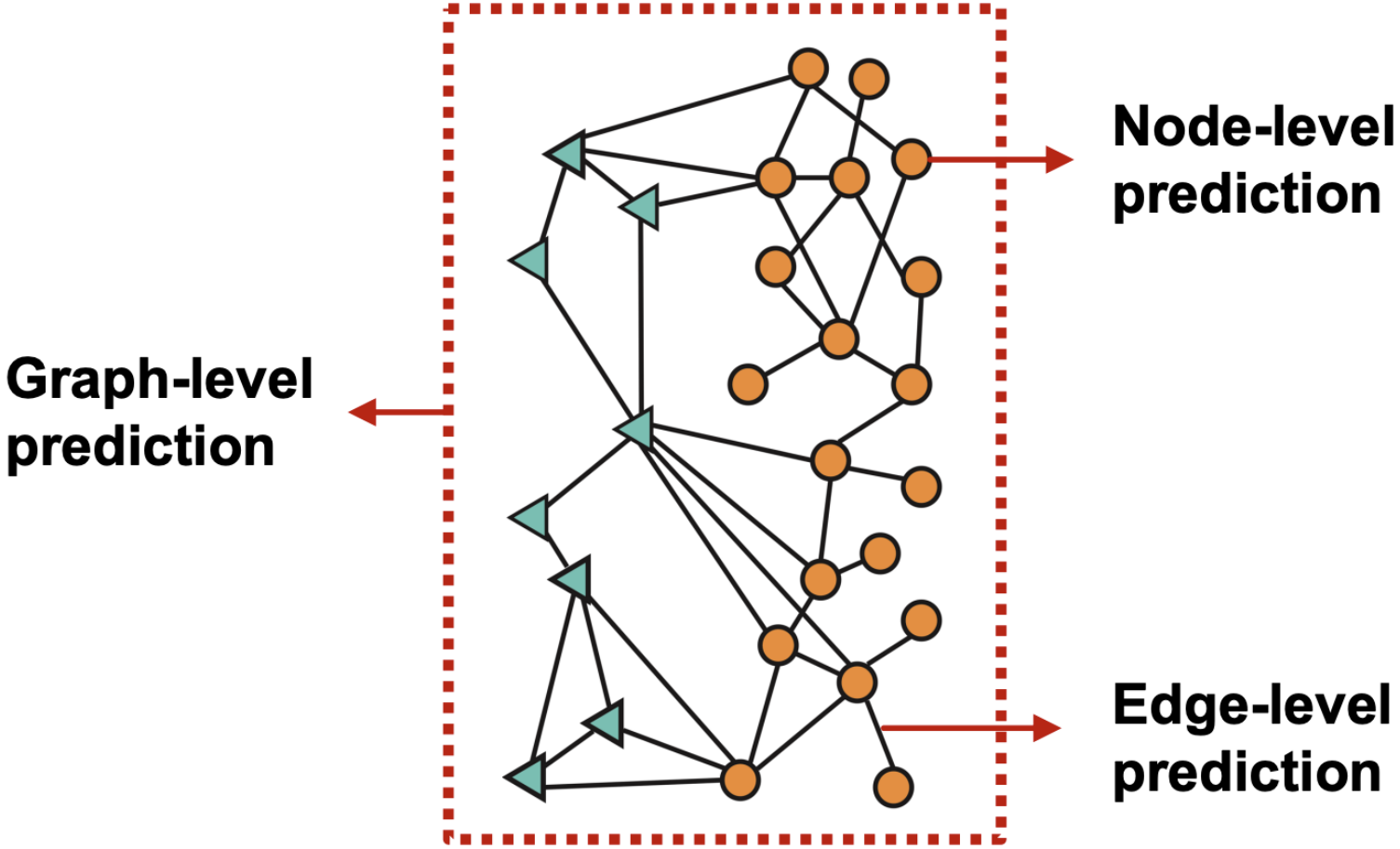


Output of a GNN: set of node embeddings

$$\{\mathbf{h}_v^{(L)}, \forall v \in G\}$$

Different tasks requires different prediction heads

Different tasks



Node-level

We can directly make prediction using node embeddings

- After GNN computation, we have d-dim node embeddings: $\left\{h_v^{(L)} \in R^d, \forall\, v \in G\right\}$
- For k-class (k-way) classification, we have two different approaches:
  1. For simple and small graphs prediction, during GNN training, we can make $d = k$ → apply softmax → **probability distribution** over k classes

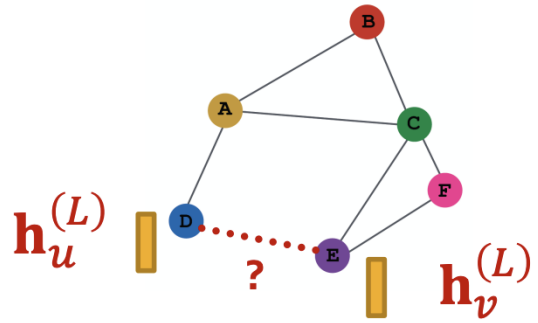
  2. Let $\hat{y}_v = Predict_{head}\left(h_v^{(L)}\right) = W^{head} h_v^{(L)}$, where $W^{head} \in R^{k \times d}$ that maps node embedding from $h_v^{(L)} \in R^d$ to $\hat{y}_v \in R^k$ → apply softmax → **probability distribution** over k classes

Edge-level

We can make prediction using pairs of node embeddings

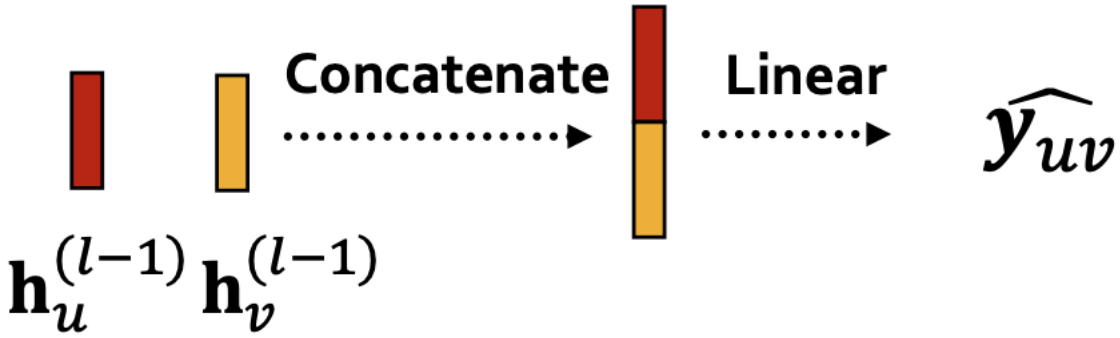
Suppose we want to make k-way prediction

$$\widehat{y_{vu}} = Predict_{head}\left(h_v^{(L)}, h_u^{(L)}\right)$$



Options for $Predict_{head}\left(h_v^{(L)}, h_u^{(L)}\right)$:

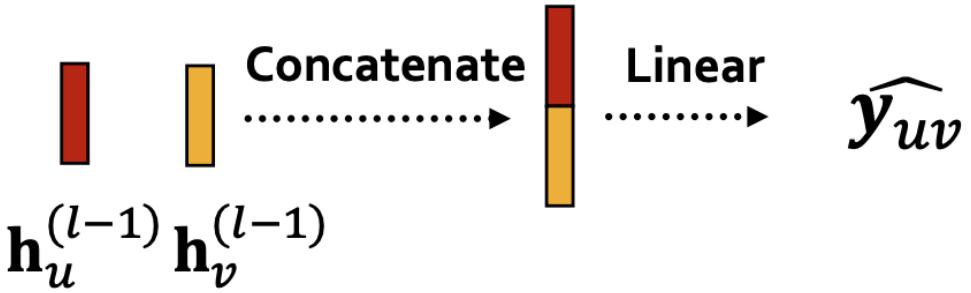
1. Concatenation + Linear



Edge-level

Options for $Predict_{head}\left(h_v^{(L)}, h_u^{(L)}\right)$:

1. Concatenation + Linear:



$\widehat{y_{vu}} = \text{Linear}(\text{Concat}(h_v^{(L)}, h_u^{(L)}))$

Here $\text{Linear}(\cdot)$ will map $2d$-dimensional embeddings to k-dim embeddings

Edge-level

Options for $Predict_{head}\left(h_v^{(L)}, h_u^{(L)}\right)$:

2. Dot product:

$$\widehat{y_{vu}} = (h_v^{(L)})^T h_u^{(L)}$$

a. **This approach only applies to 1-way prediction** (e.g., link prediction: predict the existence of an edge)

b. **Applying to k-way prediction**:

Similar to **multi-head attention**: $\boldsymbol{W}^{(1)}, \boldsymbol{W}^{(2)}, \ldots, \boldsymbol{W}^{(k)}$ trainable

$$\hat{y}_{vu}^{(1)} = (h_v^{(L)})^T \boldsymbol{W}^{(1)} h_u^{(L)}$$

$$\ldots$$

$$\hat{y}_{vu}^{(k)} = (h_v^{(L)})^T \boldsymbol{W}^{(k)} h_u^{(L)}$$

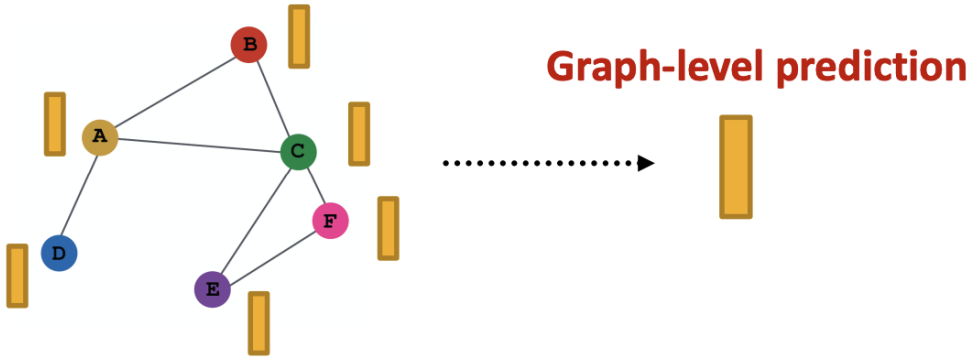
$$\widehat{y_{vu}} = Concat(\hat{y}_{vu}^{(1)}, \ldots, \hat{y}_{vu}^{(k)}) \in R^k$$

Graph-level

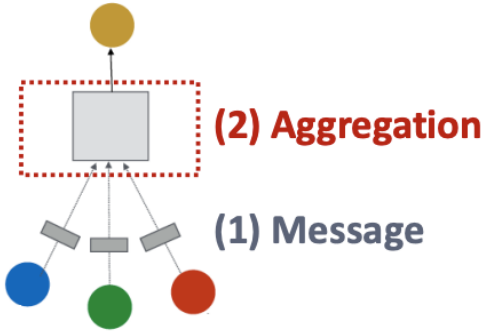
We can make predictions using all the node embeddings in our graph.

Suppose we want to make k-way prediction:

$$\widehat{y_G} = Predict_{head}(\{h_v^{(L)} \in R^d, \forall\, v \in G\})$$



$Predict_{head}$ here is similar to the aggregation part in GNN layer



Graph-level

Simple Options:

1. Global mean pooling

$$\widehat{y_G} = \boldsymbol{Mean}\left(\left\{h_v^{(L)} \in R^d, \forall\, v \in G\right\}\right)$$

2. Global max pooling

$$\widehat{y_G} = \boldsymbol{Max}\left(\left\{h_v^{(L)} \in R^d, \forall\, v \in G\right\}\right)$$

3. Global sum pooling

$$\widehat{y_G} = \boldsymbol{Sum}\left(\left\{h_v^{(L)} \in R^d, \forall\, v \in G\right\}\right)$$

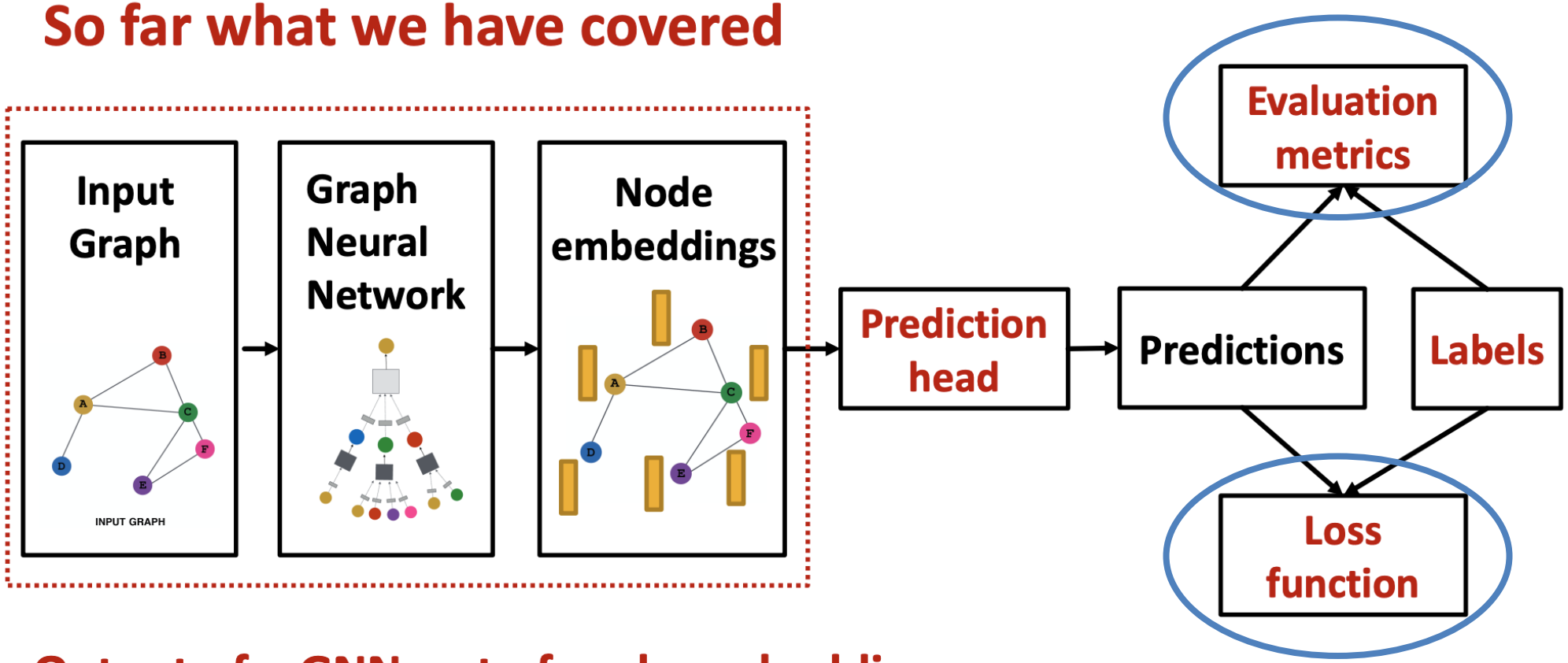
Other options:

Hierarchical Pooling, Sort Pooling, Attention-based Pooling, …

GNN Traning Pipeline

MACQUARIE University

So far what we have covered



Output of a GNN: set of node embeddings

$$\{\mathbf{h}_v^{(L)}, \forall v \in G\}$$

Classification Loss

As discussed in previous lecture, **Cross entropy (CE)** is a very common loss function in classification

K-way prediction for i-th data point, each data point can be a node/edge/graph

$$\mathrm{CE}\left(\boldsymbol{y}^{(i)}, \hat{\boldsymbol{y}}^{(i)}\right) = -\sum_{j=1}^{K} \boldsymbol{y}_j^{(i)} \log\left(\hat{\boldsymbol{y}}_j^{(i)}\right)$$

Label: $\boldsymbol{y}^{(i)}$; Prediction: $\hat{\boldsymbol{y}}^{(i)}$

i-th data point: (i)

j-th class: j

where

E.g. | 0 | 0 | 1 | 0 | 0 |

$\boldsymbol{y}^{(i)} \in \mathbb{R}^K$ = one-hot label encoding

$\hat{\boldsymbol{y}}^{(i)} \in \mathbb{R}^K$ = prediction after $\mathrm{Softmax}(\cdot)$

E.g. | 0.1 | 0.3 | 0.4 | 0.1 | 0.1 |

What is the total loss over all N training examples?

Regression Loss

For regression tasks we often use **Mean Squared Error (MSE)** a.k.a. **L2 loss**

K-way regression for data point (i):

$$\text{MSE}\left(\boldsymbol{y}^{(i)}, \hat{\boldsymbol{y}}^{(i)}\right) = \sum_{j=1}^{K} \left(\boldsymbol{y}_j^{(i)} - \hat{\boldsymbol{y}}_j^{(i)}\right)^2$$

i-th data point

j-th target

where

E.g. | 1.4 | 2.3 | 1.0 | 0.5 | 0.6 |

$\boldsymbol{y}^{(i)} \in \mathbb{R}^k$ = Real valued vector of targets

$\hat{\boldsymbol{y}}^{(i)} \in \mathbb{R}^k$ = Real valued vector of predictions

E.g. | 0.9 | 2.8 | 2.0 | 0.3 | 0.8 |

Total loss over all N training examples?

Evaluation Metric: Classification

MACQUARIE University

- Multi-class classification:

  Simply report the accuracy: $\frac{1[\arg\max(\hat{y}^{(i)})=y^{(i)}]}{N}$

- Binary Classification:

  Accuracy: $\frac{TP+TN}{N}$

  Precision: $\frac{TP}{TP+FP}$

  Recall: $\frac{TP}{TP+FN}$

  F1-score: $\frac{2P*R}{P+R}$

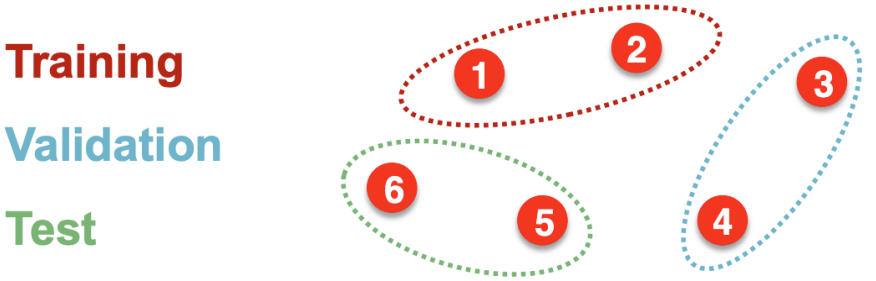
Confusion matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

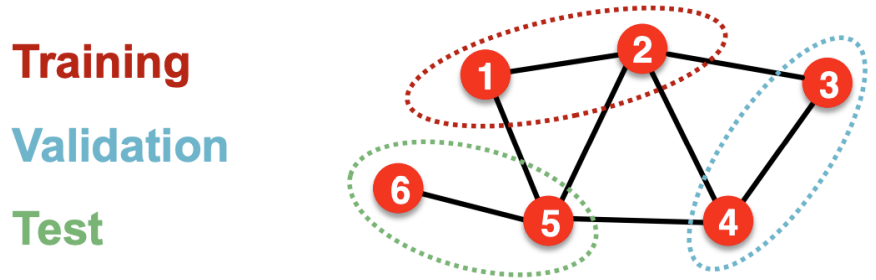
Dataset Split

MACQUARIE University

Usually in deep learning, we will randomly split our dataset into training / validation / test datasets.



However, splitting a graph dataset is different. For node-level classification, each node is a data point. **Here, data points are NOT independent**



Dataset Split

Solution 1 (Transductive setting):

The input graph can be observed in all the dataset splits. We will only split the (node) labels

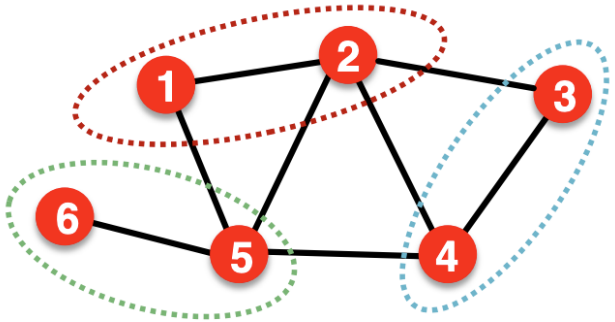
At **training time**, we compute embeddings using the entire graph, and train using node 1&2's labels

At **validation time**, we compute embeddings using the entire graph, and evaluate on node 3&4's labels

Training

Validation

Test

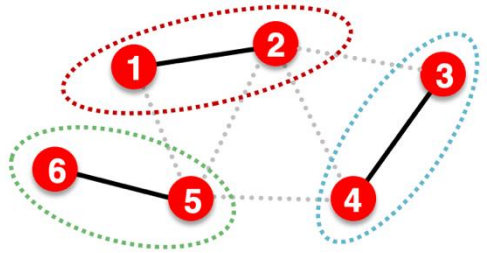


Dataset Split

Solution 2 (Inductive setting):

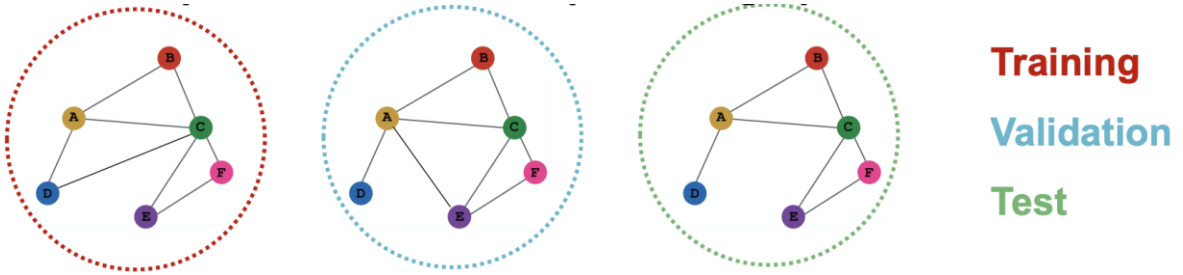
We break the edges between splits to get multiple graphs.

At training time, we compute embeddings using the graph over node 1&2, and train using node 1&2's labels.



At validation time, we compute embeddings using the graph over node 3&4, and evaluate on node 3&4's labels.

Suppose we have a dataset of 3 graphs
Each split contains an independent graph.



Dataset Split

MACQUARIE University

Transductive setting:

training / validation / test sets are on the **same graph**

The dataset consists of **one graph**

The entire graph can be observed in all dataset splits, we **only split the labels**

Only applicable to **node / edge** prediction tasks

Inductive setting:

training / validation / test sets are on **different graphs**

The dataset consists of **multiple graphs**

Each split can only observe the graph(s) within the split. A successful model should **generalize to unseen graphs**

Applicable to **node / edge / graph** tasks

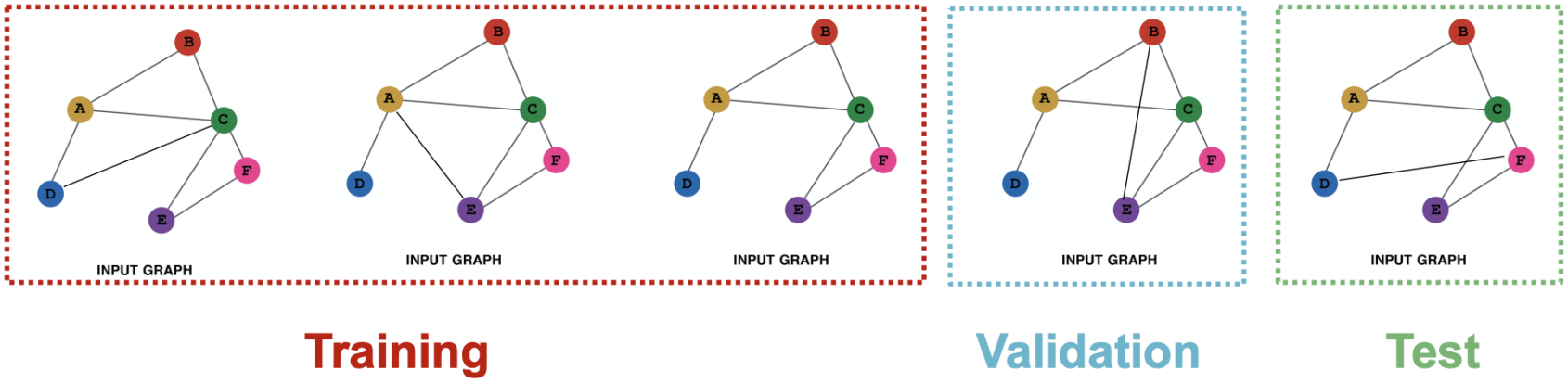
Dataset Split: Graph Classification

MACQUARIE University

Only the **inductive setting** is well defined for graph classification.

We have to test on **unseen** graphs

Suppose we have a dataset of 5 graphs. Each split will contain independent graph(s).



Example Question

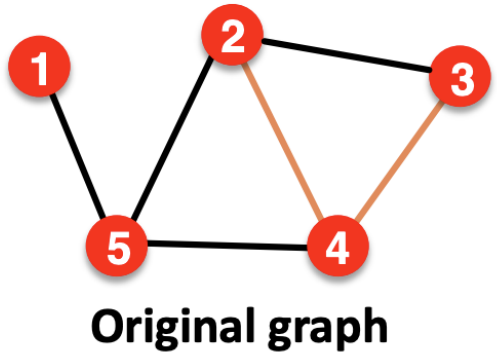
Link Prediction:

Goal: Predict missing edges

Hint:

Link prediction is unsupervised/self-supervised

We need to **create the labels** and **dataset splits** on our own

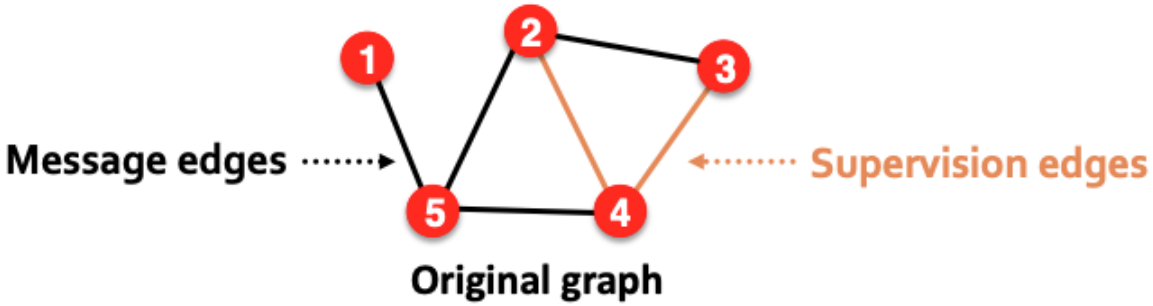


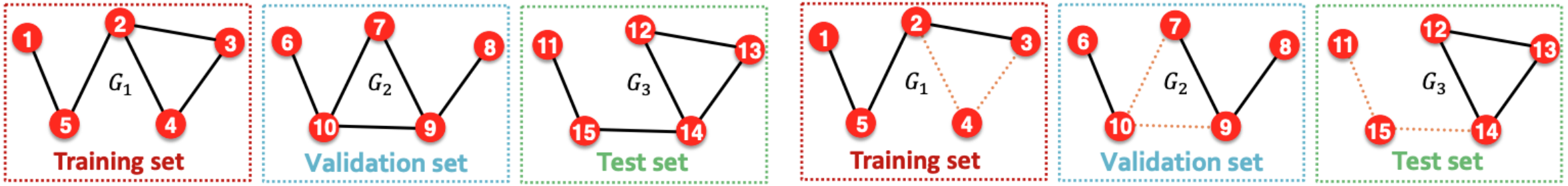
Original graph

How do you design the training pipeline?

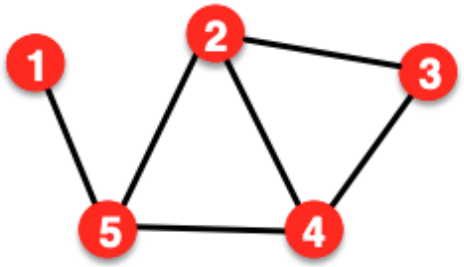
Example Question

Hide some edges from the GNN and let the GNN predict if the edges exist. Supervision edges are used as supervision for edge predictions made by the model, will not be fed into GNN!



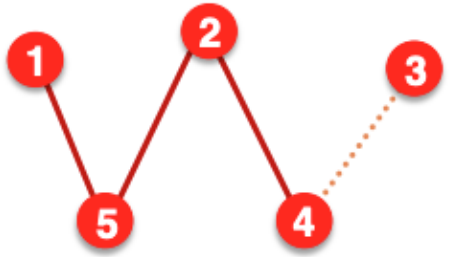


Example Question

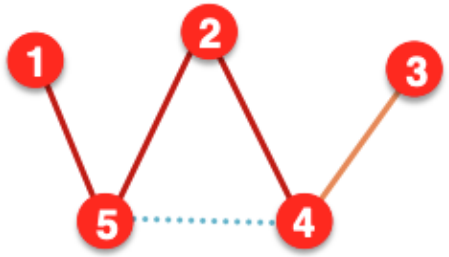


The original graph

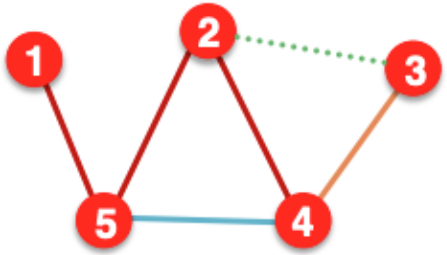
Transductive setting



(1) At training time: Use **training message edges** to predict **training supervision edges**



(2) At validation time: Use **training message edges & training supervision edges** to predict **validation edges**



(3) At test time: Use **training message edges & training supervision edges & validation edges** to predict **test edges**